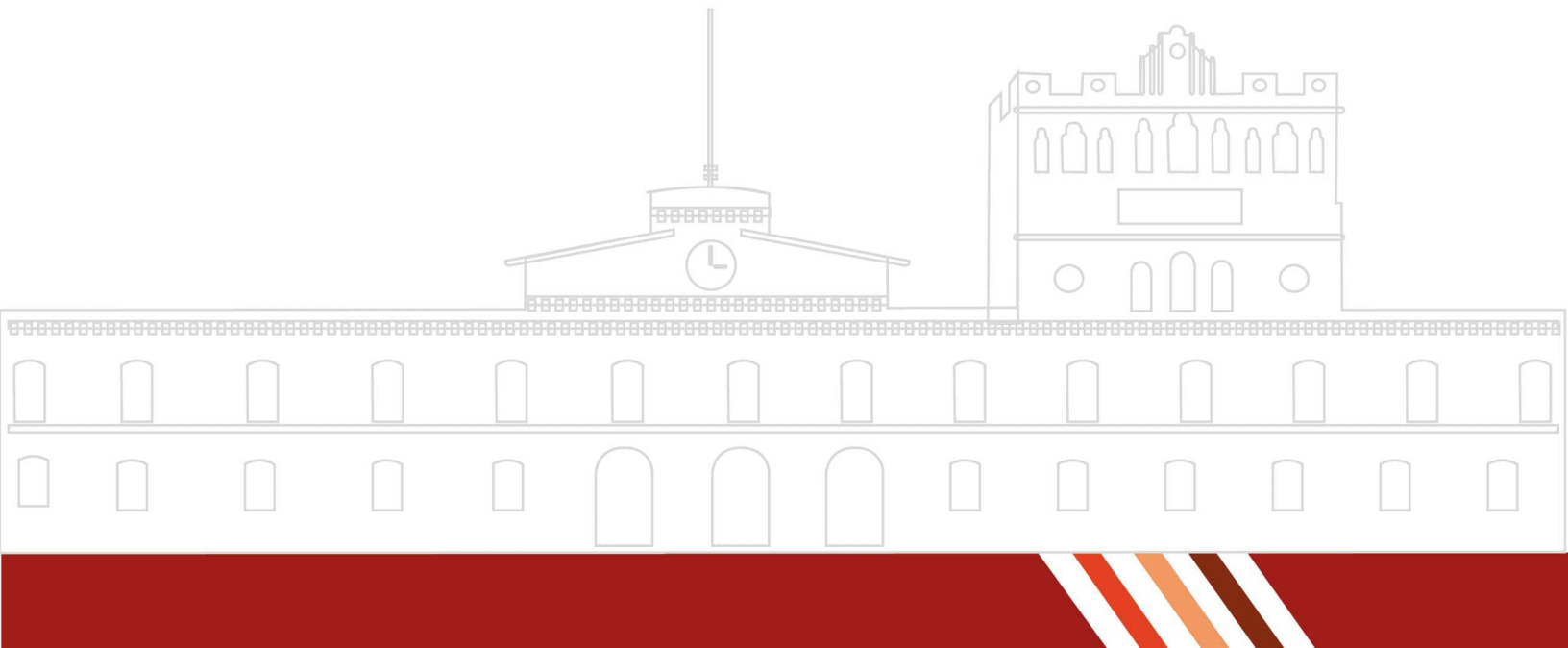


REPORTE DE PRÁCTICA NO. 1

ANÁLISIS DE EXPRESIONES REGULARES

ALUMNO:
SAMUEL ORTIZ SALINAS

Dr. Eduardo Cornejo Velázquez



1. Introducción

El análisis de cadenas de símbolos constituye una parte importante dentro del estudio de los lenguajes formales y los compiladores. Un programa fuente puede considerarse inicialmente como una secuencia de caracteres que debe ser identificada, separada y clasificada para determinar qué elementos contiene.

En esta práctica se desarrollaron dos soluciones para analizar una cadena de entrada. La primera solución se implementó mediante estructuras de datos y algoritmos que permiten recorrer el texto carácter por carácter, formar lexemas y clasificarlos según sus características. La segunda solución utiliza expresiones regulares para reconocer patrones definidos dentro de la cadena.

Las categorías identificadas por el programa son números enteros, palabras en minúsculas, palabras en mayúsculas, identificadores y símbolos. El sistema también reconoce algunos símbolos compuestos y números enteros con signo.

La práctica permite comparar ambos enfoques y comprender cómo un conjunto de caracteres puede analizarse para reconocer diferentes patrones dentro de una cadena de entrada.

2. Marco teórico

Lenguajes formales

Un lenguaje formal está formado por cadenas construidas a partir de un conjunto de símbolos denominado alfabeto. Las reglas utilizadas para determinar cuáles cadenas pertenecen al lenguaje permiten establecer patrones y estructuras válidas.

En el contexto de los compiladores, una cadena de caracteres puede analizarse para identificar unidades con significado, como números, palabras, identificadores y símbolos.

Análisis léxico

El análisis léxico corresponde al proceso de leer una secuencia de caracteres y agruparlos en unidades denominadas lexemas. Posteriormente, cada lexema puede clasificarse según el patrón que representa.

Por ejemplo, una secuencia como:

```
variable123 == -25
```

puede separarse en los siguientes lexemas:

```
variable123  ==  -25
```

Cada uno pertenece a una categoría diferente, como identificador, símbolo compuesto y número entero.

Expresiones regulares

Las expresiones regulares permiten describir patrones dentro de una cadena de texto. Mediante símbolos especiales y conjuntos de caracteres es posible definir qué combinaciones son válidas para cada categoría.

Por ejemplo:

`[0-9]+`

representa una secuencia formada por uno o más dígitos.

De forma similar:

`[a-z]+`

representa una palabra formada por letras minúsculas.

Identificadores

Un identificador es una secuencia utilizada para representar nombres dentro de un lenguaje. Para esta práctica se consideran identificadores las secuencias que pueden contener letras, dígitos y guion bajo.

Ejemplos:

`variable   dato123   _contador   HolaMundo`

3. Objetivos

Objetivo general

Implementar programas que permitan analizar cadenas de símbolos mediante el uso de estructuras de datos y algoritmos, así como mediante expresiones regulares, para identificar y clasificar diferentes tipos de lexemas.

Objetivos específicos

- Implementar un programa capaz de recorrer una cadena carácter por carácter.
- Identificar números enteros.
- Identificar palabras formadas únicamente por letras minúsculas.
- Identificar palabras formadas únicamente por letras mayúsculas.
- Identificar secuencias consideradas como identificadores.
- Reconocer símbolos individuales y símbolos compuestos.
- Definir expresiones regulares para las diferentes categorías de lexemas.
- Implementar una segunda solución utilizando la biblioteca `<regex>` de C++.
- Clasificar y contabilizar los tokens reconocidos durante el análisis.

4. Desarrollo

Análisis de requisitos

El programa desarrollado permite ingresar una cadena de texto mediante una interfaz gráfica. Posteriormente, el contenido es analizado mediante dos enfoques diferentes.

El primer enfoque utiliza estructuras de datos y algoritmos para recorrer la cadena carácter por carácter. El segundo enfoque utiliza expresiones regulares para reconocer los patrones definidos.

Las categorías reconocidas son:

- Número entero.
- Palabra en minúsculas.
- Palabra en mayúsculas.
- Identificador.
- Símbolo.

Además, se consideran símbolos compuestos como:

`==, !=, <=, >=, ++, --, +=, -=, *=, /=, &&, ||, «, »`

También se reconocen números enteros con signo, por ejemplo:

`-5    +25`

Definición del alfabeto

El alfabeto utilizado para realizar el análisis está formado por letras, dígitos, guion bajo y símbolos especiales.

De manera general:

$$\Sigma = \{0, 1, 2, \dots, 9\} \cup \{a, b, \dots, z\} \cup \{A, B, \dots, Z\} \cup \{_ \} \cup \{+, -, *, /, =, <, >, !, \&, |, ;, \dots\}$$

Los espacios, tabulaciones y saltos de línea son utilizados como separadores durante el recorrido de la cadena.

Expresiones regulares utilizadas

La Tabla 1 presenta las expresiones regulares utilizadas para representar las diferentes categorías reconocidas por el programa.

Cuadro 1: Expresiones regulares por categoría de token.

Categoría	Expresión regular
Número entero	[+-]?[0-9]+
Palabra minúscula	[a-z]+
Palabra mayúscula	[A-Z]+
Identificador	[A-Za-z_] [A-Za-z0-9_]*
Símbolo compuesto	== != <= >= ++ - += - = * = /= && << >>
Símbolo simple	Caracteres especiales individuales

Implementación mediante estructuras de datos

El primer enfoque implementado realiza el análisis mediante un recorrido secuencial de la cadena.

Inicialmente, el texto recibido se almacena en una variable de tipo `string`. Posteriormente, se obtiene la longitud de la cadena y se crea un arreglo de caracteres denominado `Buffer`. También se utiliza un arreglo denominado `Lex`, el cual permite construir temporalmente cada lexema encontrado.

El recorrido se controla mediante un índice denominado `I`. Mientras existan caracteres pendientes por analizar, el programa determina a qué tipo de secuencia pertenece el carácter actual.

Los espacios, tabulaciones y saltos de línea son ignorados.

Cuando se encuentra un carácter válido, el programa inicia la construcción de un nuevo lexema. Dependiendo del carácter encontrado, se realizan diferentes procesos.

Si dos caracteres consecutivos forman un símbolo compuesto válido, ambos se almacenan dentro del mismo lexema.

Si el carácter es `+` o `-` y el siguiente carácter es un dígito, se considera el inicio de un número con signo y se continúa leyendo hasta terminar la secuencia de dígitos.

Si el carácter es alfanumérico o un guion bajo, se continúan acumulando los caracteres consecutivos para formar una palabra, número o identificador.

Finalmente, el lexema construido es enviado a una función encargada de clasificarlo.

Diagrama de flujo del análisis

La Figura 1 representa el proceso general utilizado por el programa para recorrer la cadena, construir los lexemas, clasificarlos y almacenarlos como tokens.

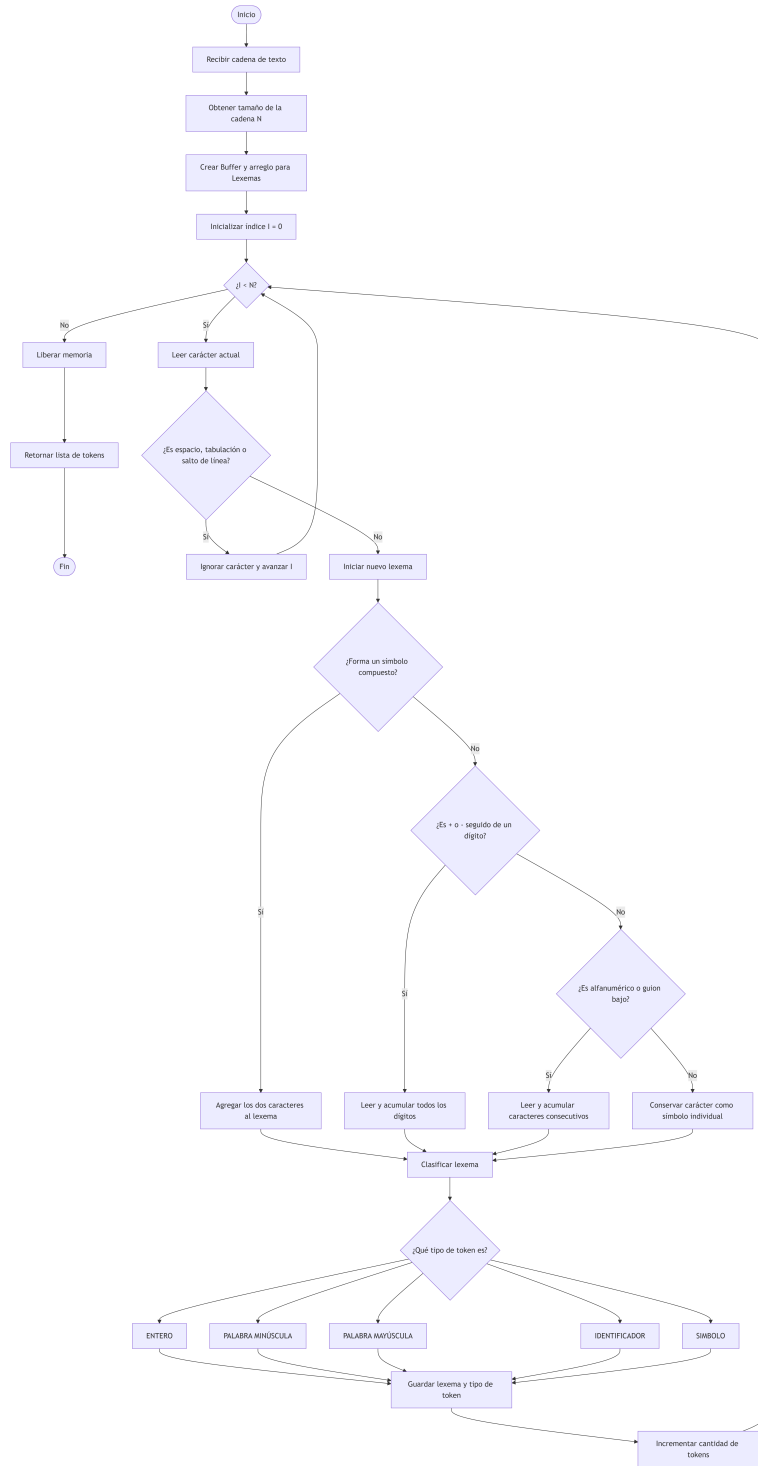


Figura 1: Diagrama de flujo del proceso de análisis de cadenas mediante estructuras de datos.

Clasificación de lexemas

Una vez construido un lexema, la función `clasificar()` analiza los caracteres que lo componen.

Durante el recorrido se determina si el lexema contiene:

- Dígitos.
- Letras minúsculas.
- Letras mayúsculas.
- Guion bajo.
- Caracteres no válidos dentro de una secuencia alfanumérica.

A partir de estas características, el programa determina el tipo de token.
Por ejemplo:

Lexema	Tipo
123	Número entero
-25	Número entero
hola	Palabra minúscula
HOLA	Palabra mayúscula
hola123	Identificador
HolaMundo	Identificador
_dato	Identificador
==	Símbolo

Fragmento de código representativo

El siguiente fragmento corresponde a la clasificación de los lexemas mediante el análisis de los caracteres que los forman.

Listing 1: Clasificación de lexemas.

```
1
2 TipoToken AutomataLexico::clasificar(const string &Lexema)
3 {
4     bool TieneDigito = false;
5     bool TieneMinuscula = false;
6     bool TieneMayuscula = false;
7     bool TieneGuion = false;
8     bool EsValido = true;
9
10    for (char C : Lexema)
11    {
12        if (isdigit(static_cast<unsigned char>(C)))
13            TieneDigito = true;
14
15        else if (islower(static_cast<unsigned char>(C)))
16            TieneMinuscula = true;
17    }
```

```

18         else if (isupper(static_cast<unsigned char>(C)))
19             TieneMayuscula = true;
20
21         else if (C == '_')
22             TieneGuion = true;
23
24         else
25             EsValido = false;
26     }
27
28     if (!EsValido)
29         return TipoToken::SIMBOLO;
30
31     if (TieneGuion ||
32         (TieneDigito &&
33          (TieneMinuscula || TieneMayuscula)) ||
34         (TieneMinuscula && TieneMayuscula))
35     {
36         return TipoToken::IDENTIFICADOR;
37     }
38
39     if (TieneDigito)
40         return TipoToken::ENTERO;
41
42     if (TieneMinuscula)
43         return TipoToken::PALABRA_MINUSCULA;
44
45     if (TieneMayuscula)
46         return TipoToken::PALABRA_MAYUSCULA;
47
48     return TipoToken::SIMBOLO;
49 }

```

Implementación mediante expresiones regulares

El segundo enfoque utiliza expresiones regulares para identificar los patrones presentes dentro de la cadena de entrada.

Las expresiones regulares permiten definir de forma compacta los patrones que representan números, palabras, identificadores y símbolos.

El proceso consiste en recorrer la cadena y extraer los elementos que coinciden con los patrones establecidos. Posteriormente, cada lexema se clasifica y se incrementa el contador correspondiente.

Este enfoque permite realizar el reconocimiento de patrones sin recorrer manualmente cada carácter mediante múltiples condiciones.

Interfaz gráfica

La aplicación cuenta con una interfaz gráfica que permite ingresar la cadena que será analizada y visualizar los resultados obtenidos.

La interfaz integra las diferentes soluciones implementadas para la práctica, permitiendo observar los lexemas encontrados, su clasificación y el conteo de cada categoría.

La Figura 2 muestra la interfaz utilizada para realizar las pruebas del programa.

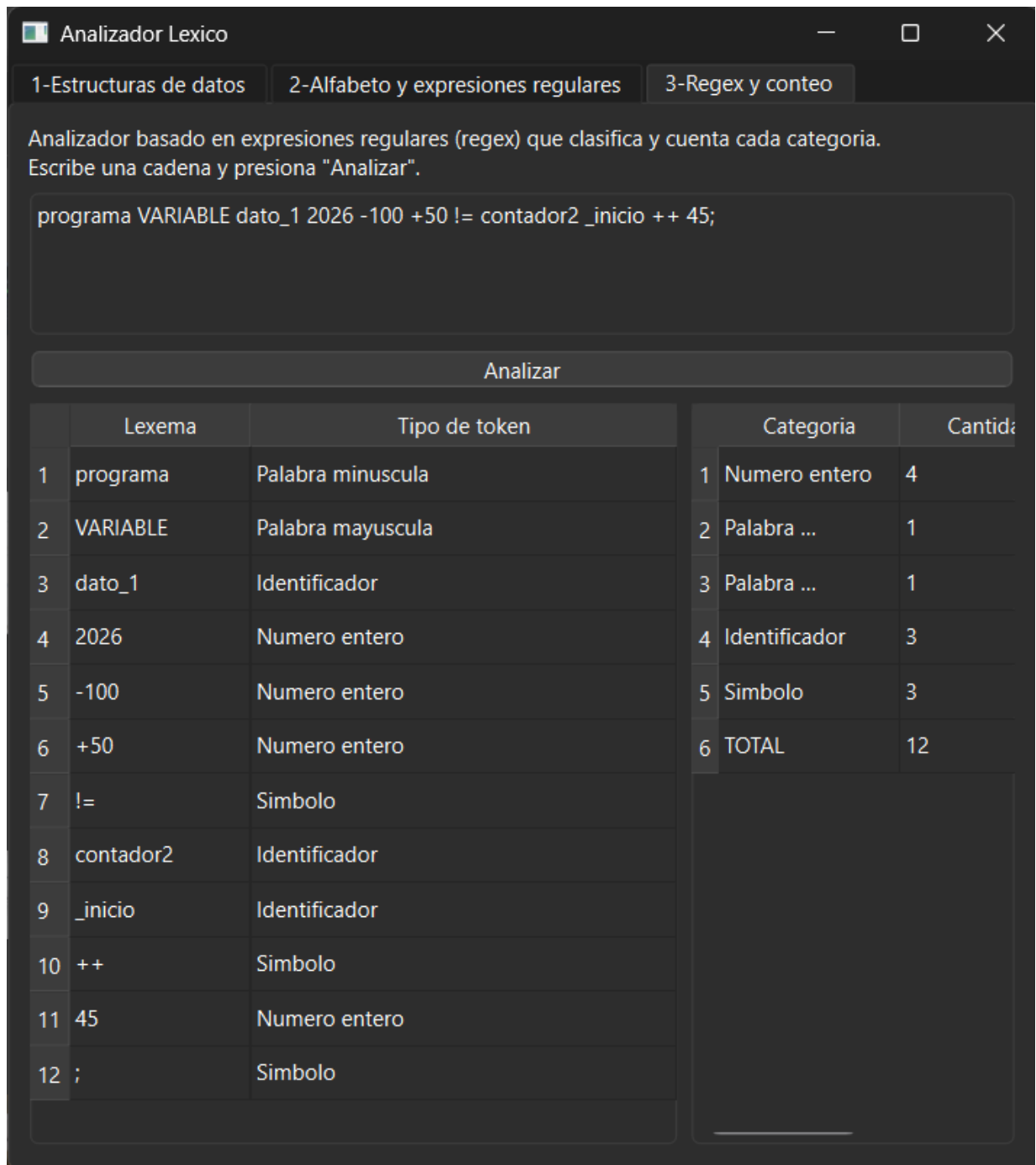


Figura 2: Interfaz gráfica del programa para el análisis de cadenas.

5. Resultados

Para comprobar el funcionamiento del programa se realizaron pruebas con diferentes combinaciones de números, palabras, identificadores y símbolos.

Una de las cadenas utilizadas para realizar pruebas fue:

`hola HOLA hola123 _dato 123 -45 == != +5;`

El programa separa los elementos encontrados y los clasifica según sus características. Los resultados esperados son los siguientes:

Lexema	Clasificación
hola	Palabra minúscula
HOLA	Palabra mayúscula
hola123	Identificador
_dato	Identificador
123	Número entero
-45	Número entero
==	Símbolo
!=	Símbolo
+5	Número entero
;	Símbolo

Las pruebas permitieron comprobar que los lexemas pueden separarse correctamente incluso cuando aparecen junto a símbolos.

Por ejemplo:

`d; → d y ;`

y:

`5; → 5 y ;`

Mientras tanto, las combinaciones consideradas válidas se mantienen como un solo lexema:

`== -5 +25`

De esta manera se comprobó el funcionamiento tanto del análisis basado en estructuras de datos como del enfoque basado en expresiones regulares.

6. Cuestionario

¿Cuál es la utilidad de las expresiones regulares en la identificación de palabras de un lenguaje?

Las expresiones regulares sirven para definir patrones y reconocer cadenas que cumplen con ciertas características. Aquí se utilizaron para poder reconocer diferentes tipos de lexema mediante analizar los caracteres que contiene

¿Cuáles elementos del lenguaje de programación utilizaste para la implementación de las estructuras de datos?, ¿cómo las utilizaste?

Utilicé C++ y elementos como `string`, arreglos de tipo `char`, estructuras, ciclos `while` y condicionales `if`. Los arreglos se utilizaron para recorrer la cadena y formar los lexemas encontrados.

¿Cuál es el proceso que seguiste para analizar las cadenas de símbolos?

Primero obtuve la cadena ingresada y recorrí cada carácter. Los espacios se usaban como separación y no se tomaban en cuenta en parte del análisis y los demás caracteres se fueron agrupando para formar lexemas. Después, cada lexema se clasificó como número, palabra, identificador o símbolo.

¿Qué elementos del lenguaje de programación te permitieron implementar expresiones regulares?, ¿qué condiciones o recomendaciones se deben seguir al escribir las expresiones regulares?

Utilicé la biblioteca `<regex>` de C++. Para escribir las expresiones regulares, definir correctamente los caracteres permitidos y considerar los símbolos especiales. También cuidando el orden de los patrones para evitar clasificaciones incorrectas.

7. Conclusion

La realización de esta práctica permitió comprender el proceso de análisis de una cadena de símbolos mediante dos enfoques diferentes.

El primer enfoque, basado en estructuras de datos y algoritmos, permitió observar directamente el recorrido carácter por carácter y el proceso mediante el cual se construyen los lexemas. Este método proporciona un mayor control sobre cada etapa del análisis.

El segundo enfoque, basado en expresiones regulares, permitió representar los patrones de reconocimiento de una forma más compacta. Las expresiones regulares simplifican la identificación de secuencias que cumplen determinadas características.

Ambas soluciones permiten resolver el problema de identificar y clasificar los elementos presentes en una cadena. Sin embargo, cada enfoque presenta una forma diferente de realizar el reconocimiento.

Finalmente, la práctica permitió aplicar conceptos relacionados con alfabetos, cadenas, patrones, estructuras de datos y expresiones regulares, así como comprender su utilidad dentro del análisis de lenguajes y la construcción de herramientas relacionadas con compiladores.

8. Bibliografía

Referencias

- [1] Carranza Sahagún, D. U., y Nolaso Salcedo, M. del C. (2024). Análisis léxico. En D. U. Carranza Sahagún (Coord.), *Compiladores: fases de análisis*. Editorial Transdigital.